LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Putri Maiza Azzura NIM. 2510817220012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Putri Maiza Azzura
NIM : 2510817220012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhammad Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 6
A. Source Code 6
B. Pembahasan 7
SOAL 2 10
A. Source Code 11
B. Output Program 13
C. Pembahasan 13
SOAL 3 15
A. Source Code 15
B. Pembahasan 16
SOAL 4 19
A. Source Code 20
B. Output Program 21
C. Pembahasan 21
TAUTAN GIT 23

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 Testcase 1 13
Gambar 2 Screenshot Hasil Jawaban Soal 2 Testcase 2 13
Gambar 3 Screenshot Hasil Jawaban Soal 2 Testcase 3 13
Gambar 4 Screenshot Hasil Jawaban Soal 4 Testcase 1 21
Gambar 5 Screenshot Hasil Jawaban Soal 4 Testcase 2 21
Gambar 6 Screenshot Hasil Jawaban Soal 4 Testcase 3 21

DAFTAR TABEL

Table 1 Source Code stack.h 6
Table 2 Source Code stack.cpp..... 6
Table 3 Source Code prob1.cpp 11
Table 4 Source Code queue.h 15
Table 5 Source Code queue.cpp 15
Table 6 Source Code prob2.cpp 20

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1 Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2 Source Code stack.cpp

```
#include "stack.h"

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}
```

```

bool isFull(const Stack* s) {
    return s->top == s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw "Stack overflow";
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw "Stack underflow";
    }
    s->top--;
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw "Stack kosong";
    }
    return *(s->top);
}
```

B. Pembahasan

Struktur data stack yang diimplementasikan pada program ini menggunakan array dengan pendekatan pointer, di mana elemen-elemen stack disimpan dalam array berukuran tetap dan posisi elemen teratas ditunjuk oleh pointer top. Stack bekerja berdasarkan prinsip Last In First Out (LIFO), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan.

Fungsi init() digunakan untuk menginisialisasi stack dalam kondisi kosong. Pada fungsi ini, pointer top diatur agar menunjuk ke satu posisi sebelum elemen pertama array, yaitu data - 1. Hal ini bertujuan agar kondisi awal stack dapat dikenali sebagai kosong. Secara logika, jika top berada sebelum awal array, maka tidak ada elemen yang tersimpan dalam stack. Oleh karena itu, kondisi ini konsisten dengan definisi stack kosong dan menjadi dasar bagi fungsi lainnya.

Fungsi isEmpty() berfungsi untuk memeriksa apakah stack dalam keadaan kosong. Fungsi ini mengembalikan nilai benar jika pointer top berada sebelum array (top < data). Hal ini benar karena pada saat stack belum diisi atau seluruh elemen telah dihapus, posisi top akan kembali ke posisi awal yaitu sebelum elemen pertama. Dengan demikian, kondisi tersebut secara tepat merepresentasikan keadaan stack kosong.

Fungsi isFull() digunakan untuk memeriksa apakah stack telah penuh. Fungsi ini akan mengembalikan nilai benar jika pointer top berada pada posisi elemen terakhir array, yaitu data + MAX - 1. Karena array memiliki kapasitas tetap sebesar MAX, maka posisi tersebut merupakan batas maksimum penyimpanan. Dengan adanya pengecekan ini sebelum operasi penambahan elemen, program dapat mencegah terjadinya overflow.

Fungsi push() digunakan untuk menambahkan elemen ke dalam stack. Algoritma yang dilakukan adalah dengan terlebih dahulu memeriksa apakah stack dalam kondisi penuh menggunakan fungsi isFull(). Jika penuh, maka program akan melempar exception menggunakan throw. Jika tidak penuh, maka pointer top akan dinaikkan satu posisi untuk menunjuk ke slot kosong berikutnya, kemudian nilai yang diberikan akan disimpan pada alamat yang ditunjuk oleh top. Proses ini menjamin bahwa elemen baru selalu ditempatkan di posisi teratas, sehingga prinsip LIFO tetap terjaga.

Fungsi pop() digunakan untuk menghapus elemen teratas dari stack. Langkah pertama yang dilakukan adalah memeriksa apakah stack kosong menggunakan fungsi isEmpty(). Jika kosong, maka akan dilempar exception karena tidak ada elemen yang dapat dihapus. Jika tidak kosong, maka pointer top cukup diturunkan satu posisi. Secara logika, elemen tidak perlu dihapus secara fisik dari array karena dengan menurunkan pointer top, elemen tersebut sudah tidak lagi dapat diakses dan dianggap keluar dari stack. Hal ini membuat operasi menjadi lebih efisien.

Fungsi peek() digunakan untuk melihat nilai elemen teratas tanpa menghapusnya. Fungsi ini juga melakukan pengecekan apakah stack kosong. Jika kosong, maka exception akan dilempar. Jika tidak, maka nilai yang ditunjuk oleh pointer top akan dikembalikan. Karena top selalu menunjuk ke elemen terakhir yang dimasukkan, maka nilai tersebut adalah elemen teratas stack sesuai dengan prinsip LIFO.

Dari segi pembuktian kebenaran algoritma, seluruh operasi pada stack ini mempertahankan invariant bahwa pointer top selalu menunjuk ke elemen teratas atau berada sebelum array

jika stack kosong. Operasi push selalu menaikkan top sebelum menyimpan nilai, sedangkan pop selalu menurunkan top setelah penghapusan elemen. Dengan demikian, urutan elemen dalam stack tetap terjaga dan sesuai dengan prinsip LIFO.

Berdasarkan analisis kompleksitas, seluruh operasi pada stack ini memiliki kompleksitas waktu O(1) karena setiap fungsi hanya melakukan operasi sederhana tanpa perulangan atau rekursi. Fungsi seperti init, isEmpty, dan isFull hanya melakukan operasi perbandingan, sedangkan push, pop, dan peek hanya melibatkan manipulasi pointer dan akses elemen secara langsung. Dari sisi kompleksitas ruang, semua fungsi juga memiliki kompleksitas O(1) karena tidak menggunakan memori tambahan selain variabel yang sudah ada. Dengan demikian, implementasi stack ini tergolong efisien baik dari segi waktu maupun ruang.

Soal ini hanya berfokus pada implementasi fungsi sehingga tidak menghasilkan output secara langsung, kecuali dilakukan pengujian melalui program utama.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru
- dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut,
- lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.
Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input

- Simbol yang valid hanya:
    - bilangan bulat
    - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 3 Source Code prob1.cpp

```
#include <iostream>
#include <sstream>
#include "stack.h"
using namespace std;

int main() {
    int n;
    cin >> n;
    cin.ignore();

    string input;
```

```
    getline(cin, input);

    stringstream ss(input);
    string token;

    Stack s;
    init(&s);

    try {
        while (ss >> token) {
            // angka
            if (isdigit(token[0]) || token.size() > 1) {
                push(&s, stoi(token));
            } else {
                int b = peek(&s); pop(&s);
                int a = peek(&s); pop(&s);

                int hasil;

                if (token == "+") hasil = a + b;
                else if (token == "-") hasil = a - b;
                else if (token == "*") hasil = a * b;
                else if (token == "/") hasil = a / b;

                push(&s, hasil);
            }
        }

        cout << peek(&s) << endl;

    } catch (const char* err) {
        cout << err << endl;
    }

    return 0;
}
```

B. Output Program

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
3
2 3 +
5
```

Gambar 1 Screenshot Hasil Jawaban Soal 2 Testcase 1

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
9
5 1 2 + 4 * + 3 -
14
```

Gambar 2 Screenshot Hasil Jawaban Soal 2 Testcase 2

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
5
10 2 / 3 *
15
```

Gambar 3 Screenshot Hasil Jawaban Soal 2 Testcase 3

C. Pembahasan

Program pada soal ini bertujuan untuk menghitung hasil dari suatu ekspresi yang dituliskan dalam bentuk postfix atau Reverse Polish Notation (RPN). Pada metode ini, operator ditulis setelah operand, sehingga tidak diperlukan tanda kurung untuk menentukan prioritas operasi. Untuk menyelesaikan permasalahan ini, digunakan struktur data stack karena sifatnya yang sesuai dengan mekanisme perhitungan postfix, yaitu Last In First Out (LIFO).

Algoritma dimulai dengan membaca jumlah simbol (n) dan deretan simbol yang terdiri dari bilangan bulat maupun operator. Program kemudian memproses setiap simbol dari kiri ke kanan menggunakan perulangan. Jika simbol yang dibaca merupakan bilangan, maka nilai tersebut langsung dimasukkan ke dalam stack menggunakan operasi push. Hal ini dilakukan karena bilangan tersebut akan digunakan dalam operasi berikutnya.

Sebaliknya, jika simbol yang dibaca merupakan operator, maka program akan mengambil dua nilai teratas dari stack menggunakan operasi peek dan pop. Nilai pertama yang diambil merupakan operand kedua (b), dan nilai berikutnya merupakan operand pertama (a). Setelah itu, dilakukan operasi aritmatika sesuai dengan operator yang dibaca, seperti penjumlahan, pengurangan, perkalian, atau pembagian. Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam stack. Proses ini terus berlanjut hingga seluruh simbol selesai diproses.

Setelah semua simbol diproses, akan tersisa satu nilai di dalam stack yang merupakan hasil akhir dari ekspresi tersebut. Nilai ini kemudian ditampilkan sebagai output program. Penggunaan stack dalam algoritma ini terbukti benar karena setiap operator selalu bekerja pada dua operand terakhir yang dimasukkan, sesuai dengan prinsip LIFO. Hal ini menjamin bahwa urutan operasi tetap sesuai dengan aturan postfix.

Dari segi kompleksitas, algoritma ini memiliki kompleksitas waktu O(n), di mana n adalah jumlah simbol dalam ekspresi. Hal ini karena setiap simbol hanya diproses satu kali dalam perulangan. Untuk kompleksitas ruang, algoritma ini juga memiliki kompleksitas O(n), karena dalam kasus terburuk stack dapat menyimpan hingga n elemen jika semua simbol adalah bilangan. Dengan demikian, solusi ini efisien dan sesuai untuk batasan yang diberikan pada soal.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 4 Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Table 5 Source Code queue.cpp

```
#include "queue.h"

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data;
}
```

```
 7
 8 bool isEmpty(const Queue* q) {
 9     return q->front == q->rear;
10 }
11
12 bool isFull(const Queue* q) {
13     return q->rear == q->data + MAX;
14 }
15
16 void enqueue(Queue* q, int value) {
17     if (isFull(q)) {
18         throw "Queue overflow";
19     }
20     *(q->rear) = value;
21     q->rear++;
22 }
23
24 void dequeue(Queue* q) {
25     if (isEmpty(q)) {
26         throw "Queue underflow";
27     }
28     q->front++;
29 }
30
31 int front(const Queue* q) {
32     if (isEmpty(q)) {
33         throw "Queue kosong";
34     }
35     return *(q->front);
36 }
37
38 int back(const Queue* q) {
39     if (isEmpty(q)) {
40         throw "Queue kosong";
41     }
42     return *(q->rear - 1);
43 }
```

B. Pembahasan

Struktur data queue yang diimplementasikan pada program ini menggunakan array dengan pendekatan pointer, di mana elemen-elemen disimpan secara berurutan dalam array dan diakses menggunakan dua pointer, yaitu front dan rear. Queue bekerja berdasarkan prinsip First In First Out (FIFO), yaitu elemen yang pertama kali dimasukkan akan menjadi elemen pertama yang dikeluarkan.

Fungsi init() digunakan untuk menginisialisasi queue dalam keadaan kosong. Pada fungsi ini, pointer front dan rear diatur untuk menunjuk ke alamat awal array (data). Hal ini menunjukkan bahwa belum ada elemen yang disimpan. Secara logika, kondisi ini benar karena jika front dan rear berada pada posisi yang sama, maka queue tidak memiliki elemen.

Fungsi isEmpty() berfungsi untuk memeriksa apakah queue kosong. Fungsi ini mengembalikan nilai benar jika pointer front sama dengan rear. Hal ini dapat dibuktikan karena setiap elemen yang dimasukkan akan menaikkan rear, dan setiap elemen yang dikeluarkan akan menaikkan front. Jika keduanya kembali sama, berarti semua elemen telah diproses dan queue kembali kosong.

Fungsi isFull() digunakan untuk memeriksa apakah queue telah mencapai kapasitas maksimum. Kondisi penuh terjadi ketika pointer rear mencapai batas akhir array, yaitu data + MAX. Karena queue ini menggunakan array statis tanpa pergeseran ulang atau circular queue, maka setelah rear mencapai batas tersebut, tidak ada lagi ruang untuk menambahkan elemen baru, meskipun sebelumnya telah dilakukan operasi dequeue.

Fungsi enqueue() digunakan untuk menambahkan elemen ke dalam queue. Algoritma dimulai dengan memeriksa apakah queue penuh menggunakan fungsi isFull(). Jika penuh, maka akan dilempar exception sebagai bentuk penanganan error. Jika tidak penuh, maka nilai akan disimpan pada alamat yang ditunjuk oleh rear, kemudian pointer rear dinaikkan satu posisi. Hal ini memastikan bahwa elemen baru selalu ditambahkan di bagian belakang queue sesuai prinsip FIFO.

Fungsi dequeue() digunakan untuk menghapus elemen dari bagian depan queue. Langkah pertama adalah memeriksa apakah queue kosong menggunakan fungsi isEmpty(). Jika kosong, maka akan dilempar exception. Jika tidak kosong, maka pointer front cukup dinaikkan satu posisi. Secara logika, elemen tidak perlu dihapus secara fisik dari array karena dengan memajukan pointer front, elemen tersebut sudah tidak lagi dapat diakses.

Fungsi front() digunakan untuk mengambil nilai elemen paling depan tanpa menghapusnya. Fungsi ini akan mengembalikan nilai yang ditunjuk oleh pointer front. Karena front selalu menunjuk ke elemen pertama yang belum dikeluarkan, maka nilai tersebut adalah elemen terdepan queue.

Fungsi back() digunakan untuk mengambil nilai elemen paling belakang tanpa menghapusnya. Fungsi ini mengembalikan nilai pada posisi rear - 1, karena pointer rear

selalu menunjuk ke posisi kosong berikutnya setelah elemen terakhir. Oleh karena itu, elemen terakhir berada tepat satu posisi sebelum rear.

Dari segi pembuktian kebenaran algoritma, seluruh operasi menjaga invariant bahwa elemen berada di antara pointer front dan rear. Operasi enqueue selalu menambah elemen di belakang dengan menaikkan rear, sedangkan dequeue menghapus elemen di depan dengan menaikkan front. Dengan demikian, urutan elemen tetap terjaga sesuai prinsip FIFO.

Berdasarkan analisis kompleksitas, seluruh operasi pada queue ini memiliki kompleksitas waktu O(1) karena setiap fungsi hanya melakukan operasi sederhana tanpa perulangan. Fungsi seperti init, isEmpty, dan isFull hanya melakukan perbandingan pointer, sedangkan enqueue, dequeue, front, dan back hanya melibatkan akses langsung dan manipulasi pointer. Dari sisi kompleksitas ruang, semua fungsi juga memiliki kompleksitas O(1) karena tidak menggunakan memori tambahan selain array yang sudah dialokasikan. Dengan demikian, implementasi queue ini efisien dalam penggunaan waktu dan ruang.

Soal ini hanya berfokus pada implementasi fungsi sehingga tidak menghasilkan output secara langsung, kecuali dilakukan pengujian melalui program utama.

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 6 Source Code prob2.cpp

```
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int arr[100];

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }

    cout << sum;

    for (int i = k; i < n; i++) {
        sum = sum - arr[i - k] + arr[i];
        cout << " " << sum;
    }

    return 0;
}
```

B. Output Program

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
5 3
1 2 3 4 5
6 9 12
```

Gambar 4 Screenshot Hasil Jawaban Soal 4 Testcase 1

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
5 5
1 2 3 4 5
15
```

Gambar 5 Screenshot Hasil Jawaban Soal 4 Testcase 2

```
USER@Putri MINGW64
e-2ZURE (main)
$ ./prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 6 Screenshot Hasil Jawaban Soal 4 Testcase 3

C. Pembahasan

Program pada soal ini bertujuan untuk menghitung jumlah dari setiap kelompok elemen berurutan dalam suatu array dengan ukuran tertentu (k). Pendekatan yang digunakan adalah metode sliding window, yaitu teknik optimasi yang memungkinkan perhitungan jumlah dilakukan secara efisien tanpa harus menjumlahkan ulang seluruh elemen pada setiap langkah.

Algoritma dimulai dengan membaca jumlah elemen (n) dan ukuran jendela (k), kemudian seluruh elemen array disimpan. Langkah pertama adalah menghitung jumlah dari k elemen

pertama sebagai jendela awal. Nilai ini kemudian langsung ditampilkan sebagai hasil pertama.

Selanjutnya, untuk menghitung jumlah jendela berikutnya, program tidak lagi menjumlahkan seluruh elemen dari awal. Sebagai gantinya, digunakan teknik sliding window dengan memanfaatkan hasil sebelumnya. Caranya adalah dengan mengurangi elemen yang keluar dari jendela dan menambahkan elemen baru yang masuk. Secara matematis, jika jumlah sebelumnya adalah sum, maka jumlah berikutnya dapat dihitung dengan rumus: sum = sum - elemen lama + elemen baru. Dengan cara ini, setiap pergeseran jendela hanya membutuhkan satu operasi pengurangan dan satu operasi penjumlahan.

Proses ini dilakukan secara berulang dengan menggeser jendela satu langkah ke kanan hingga tidak memungkinkan lagi untuk membentuk kelompok baru berukuran k. Setiap hasil perhitungan dicetak dalam satu baris dengan dipisahkan oleh spasi sesuai dengan format output yang diminta.

Dari segi pembuktian, algoritma ini benar karena setiap jendela selalu merepresentasikan k elemen yang berurutan, dan proses update jumlah selalu mempertahankan nilai yang sesuai dengan elemen dalam jendela tersebut. Dengan demikian, tidak ada elemen yang terlewat atau dihitung lebih dari sekali.

Dari segi kompleksitas, algoritma ini memiliki kompleksitas waktu O(n), karena setiap elemen hanya diproses satu kali setelah perhitungan awal. Jika menggunakan metode biasa, kompleksitas bisa mencapai O(n × k), sehingga metode sliding window jauh lebih efisien. Untuk kompleksitas ruang, program menggunakan O(n) untuk menyimpan array dan O(1) untuk variabel tambahan.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-2ZURE